1. Introduction

   這次的深度學習 lab，我先計算了神經網路中各層的 back propagation 需要用到的偏微分，使用 chain rule 湊出需要的變數，再寫成程式碼。程式碼中，不能使用深度學習套件，只能使用 numpy 和標準函式庫，需要對兩種資料實作前向傳播與反向傳播，不斷重複更新權重，直到最後模型收斂。另外，我也嘗試改變不同的參數與使用的函數，發現對訓練速度和準確率都有影響。

2. Implementation Details

   A. Sigmoid function

      在程式碼中，我依照說明 pdf 製作 Sigmoid function，做為 Activation function。Sigmoid(x) = 1.0 / (1.0 + np.exp(-x))，能把負無限大到正無限大之間的值壓縮到 0~1 之間，適合用於二元分類問題。微分則是 np.multiply(x, 1.0 - x)，其中的 x 為 sigmoid 後的數值。

   B. Neural network architecture

      我建了(2,3,3,1)的神經網路，總共有四層，分別為輸入層、隱藏層、隱藏層、輸出層，其中輸入層有 2 個神經元、隱藏層都是 3 個神經元、輸出層有 1 個神經元。各層間會使用 weight 與 bias 進行矩陣運算，最後再使用 sigmoid function。

   C. Backpropagation

      先進行 forward，計算乘以 weight 並加上 bias 後的 p1，再使用 sigmoid 函數變成 z1，重複多次得到最終的輸出。接著再進行 backward，計算出各個 delta 方便其他層使用，同時利用 chain rule，把 delta 乘以輸入值計算 gradient。在算出所有的 gradient 後，使用梯度下降，把 weight 減掉一部份的 gradient，把 bias 減掉一部份的 delta 平均。不斷反覆運作這樣的步驟，直到 100000 回合。其中設定的學習率為 0.1。

3. Experimental Results

   A. Screenshot and comparison figure

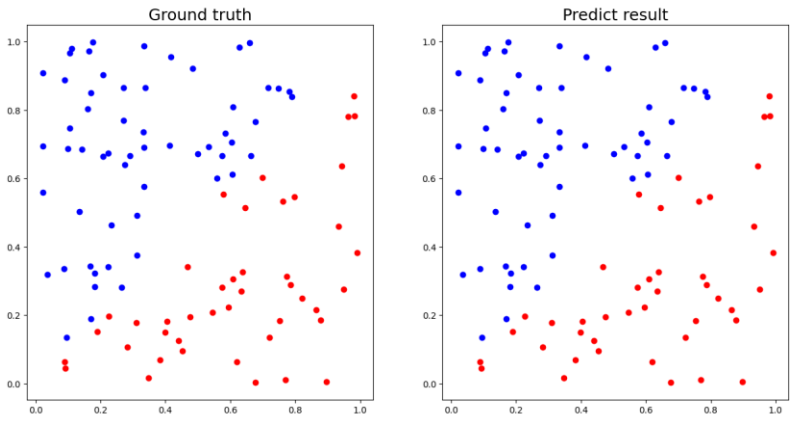
linear	XOR

```
epoch 0 loss : 0.25017769670565276
epoch 5000 loss : 0.246219974204331082
epoch 10000 loss : 0.1811706073847662
epoch 15000 loss : 0.046055627752680725
epoch 20000 loss : 0.030454141580456312
epoch 25000 loss : 0.02441648022692711
epoch 30000 loss : 0.020968961266702658
epoch 35000 loss : 0.018662107491058254
epoch 40000 loss : 0.016972382440379793
epoch 45000 loss : 0.015655208260251477
epoch 50000 loss : 0.014580322628756009
epoch 55000 loss : 0.013672636438684788
epoch 60000 loss : 0.012886374671189067
epoch 65000 loss : 0.012192378998443292
epoch 70000 loss : 0.011571369427865696
epoch 75000 loss : 0.011010152903603411
epoch 80000 loss : 0.010499389532225106
epoch 85000 loss : 0.010032228767987377
epoch 90000 loss : 0.009603454648988846
epoch 95000 loss : 0.009208940477150518
```

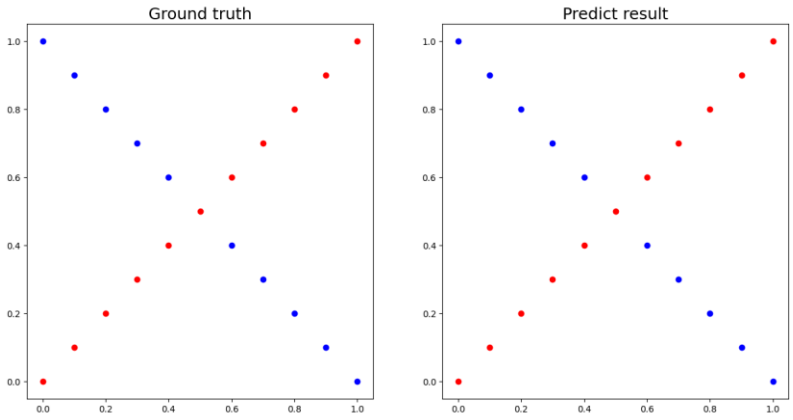
```
epoch 0 loss : 0.29867741383676855
epoch 5000 loss : 0.24788823701392063
epoch 10000 loss : 0.24763378599241537
epoch 15000 loss : 0.24742434755084317
epoch 20000 loss : 0.24718887808485132
epoch 25000 loss : 0.2468476294678547
epoch 30000 loss : 0.24622885518418489
epoch 35000 loss : 0.24480540375420703
epoch 40000 loss : 0.24065401870179692
epoch 45000 loss : 0.22871900017843808
epoch 50000 loss : 0.20198771946472915
epoch 55000 loss : 0.1424046583003785
epoch 60000 loss : 0.08785378203522957
epoch 65000 loss : 0.053487400852867725
epoch 70000 loss : 0.03478148608128957
epoch 75000 loss : 0.02459505472245265
epoch 80000 loss : 0.01809509705335182
epoch 85000 loss : 0.01358981371451093
epoch 90000 loss : 0.010377821763651485
epoch 95000 loss : 0.00806516761023407
```

依照說明的 pdf 檔案，每 5000 個 epoch 輸出一次 loss，並且截圖

linear



XOR



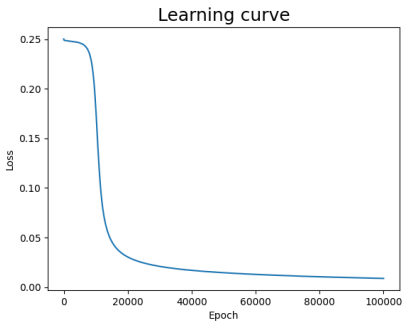
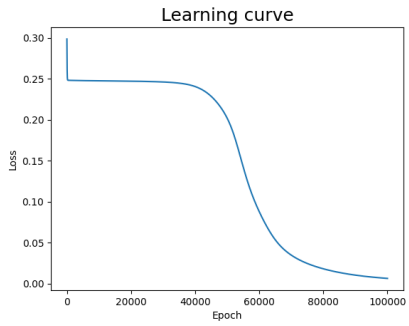
兩個表格皆為左側 Ground truth，右側 Predict result，不管是 linear 還是 XOR，左圖都與右圖分部完全相同。

B. Show the accuracy of your prediction

linear	XOR
Iter91 | Ground truth: 0 | prediction: 0.00151 | Iter92 | Ground truth: 1 | prediction: 0.99981 | Iter93 | Ground truth: 1 | prediction: 0.99971 | Iter94 | Ground truth: 1 | prediction: 0.99857 | Iter95 | Ground truth: 0 | prediction: 0.00044 | Iter96 | Ground truth: 1 | prediction: 0.99975 | Iter97 | Ground truth: 0 | prediction: 0.00042 | Iter98 | Ground truth: 0 | prediction: 0.00099 | Iter99 | Ground truth: 1 | prediction: 0.74630 | loss=0.00495 accuracy=100.00%	Iter 0 | Ground truth: 0 | prediction: 0.03303 | Iter 1 | Ground truth: 1 | prediction: 0.98964 | Iter 2 | Ground truth: 0 | prediction: 0.01021 | Iter 3 | Ground truth: 1 | prediction: 0.99147 | Iter 4 | Ground truth: 0 | prediction: 0.01534 | Iter 5 | Ground truth: 1 | prediction: 0.99064 | Iter 6 | Ground truth: 0 | prediction: 0.01204 | Iter 7 | Ground truth: 1 | prediction: 0.95724 | Iter 8 | Ground truth: 0 | prediction: 0.10417 | Iter 9 | Ground truth: 1 | prediction: 0.83491 | Iter10 | Ground truth: 0 | prediction: 0.17398 | Iter11 | Ground truth: 0 | prediction: 0.13829 | Iter12 | Ground truth: 1 | prediction: 0.79382 | Iter13 | Ground truth: 0 | prediction: 0.01028 | Iter14 | Ground truth: 1 | prediction: 0.98614 | Iter15 | Ground truth: 0 | prediction: 0.00509 | Iter16 | Ground truth: 1 | prediction: 0.99708 | Iter17 | Ground truth: 0 | prediction: 0.00139 | Iter18 | Ground truth: 1 | prediction: 0.99777 | Iter19 | Ground truth: 0 | prediction: 0.00062 | Iter20 | Ground truth: 1 | prediction: 0.99818 | loss=0.00638 accuracy=100.00%

Linear 和 XOR 的測試結果都是 100%的準確率

C. Learning curve

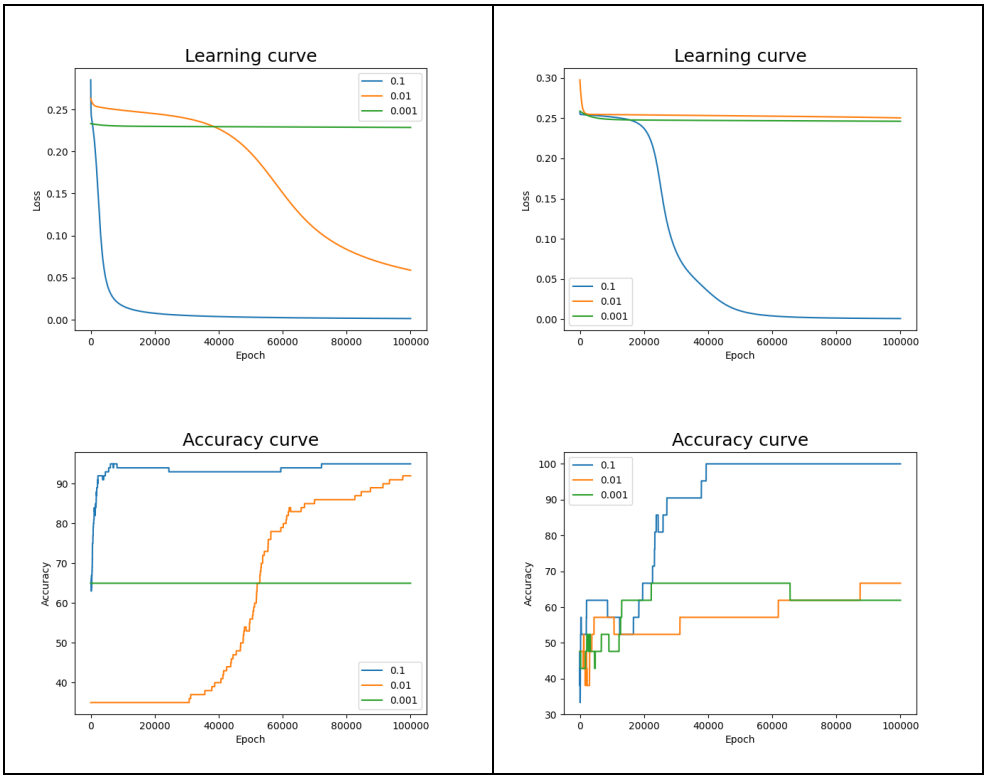
linear	XOR
	

Linear 收斂較快，大約在 20000~40000 epoch 後就收斂好了，XOR 中間遇到了瓶頸，後來才繼續收斂

4. Discussion

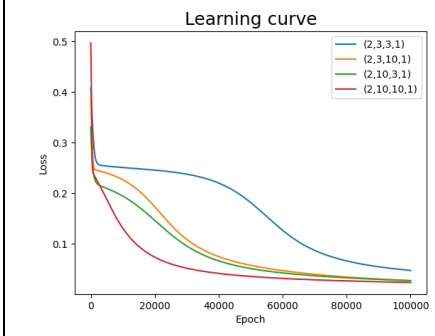
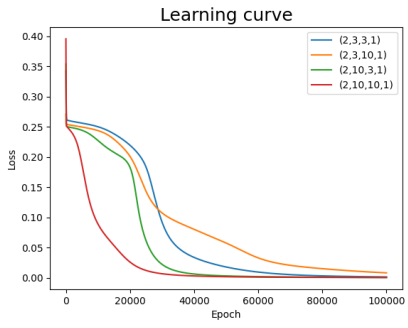
A. Try different learning rates

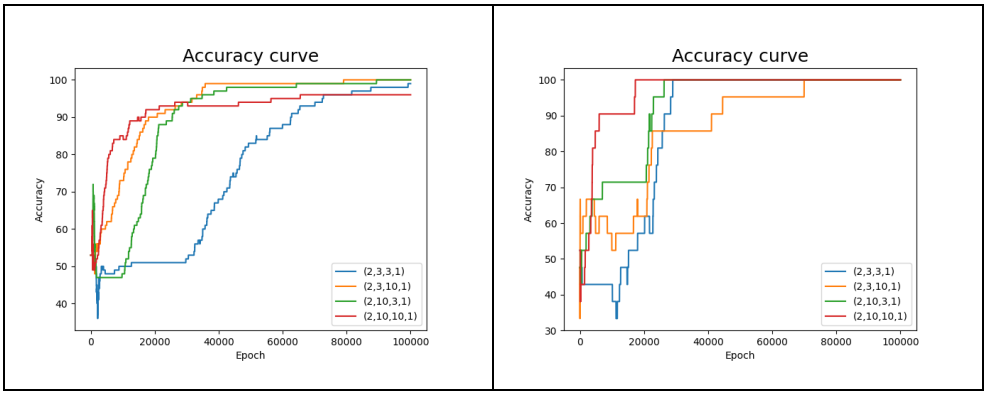
linear	XOR



我測試了三種不同的學習率 0.1, 0.01, 0.001，由圖中可以發現 0.1 學習最快，準確率也最高。在 linear 中，0.01 表現地比 0.001 好，而 XOR 則差不多。

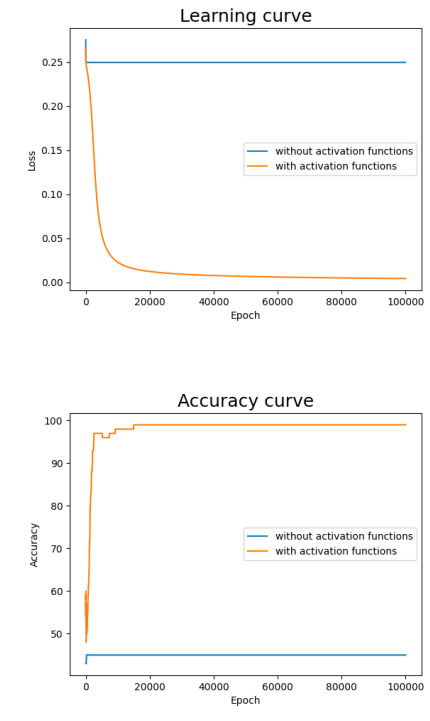
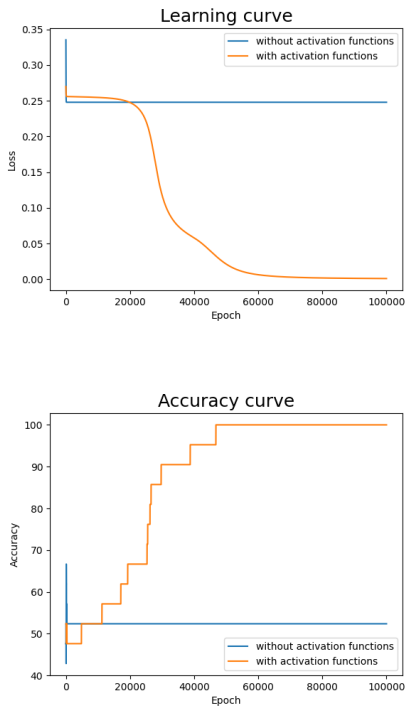
B. Try different numbers of hidden units

linear	XOR
	



我測試了四種神經網路架構，包括(2,3,3,1), (2,3,10,1), (2,10,3,1), (2,10,10,1)，其中(2,10,10,1)雖然收斂最快，最終的準確率卻不是最高的，可能是因為數據只有 100 筆，節點太多容易過擬合。而(2,3,10,1)和(2,10,3,1)的總節點數量雖然相同，但表現出來的結果卻不同，所以層的排列順序也會影響到結果。

C. Try without activation functions

linear	XOR
	

如果不使用 activation functions 就沒辦法收斂，準確率極低。

5. Questions

A. What is the purpose of activation functions?

activation functions 是為了讓神經網路中有非線性的關係，因為如果沒加入，整個神經網路可以透過化簡變成單一的矩陣運算。加入 activation functions 才具備處理複雜問題的能力。

B. What might happen if the learning rate is too large or too small?

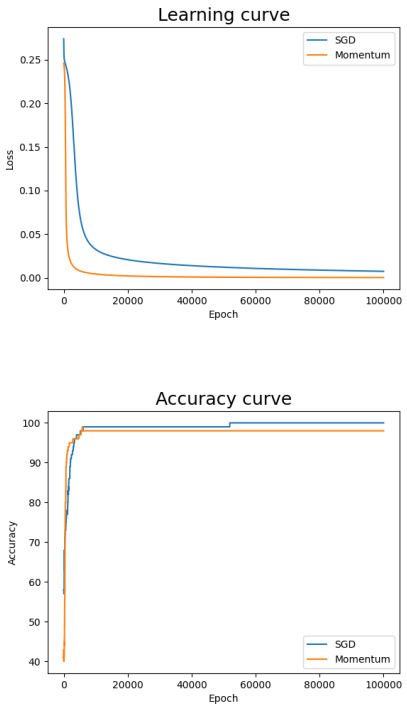
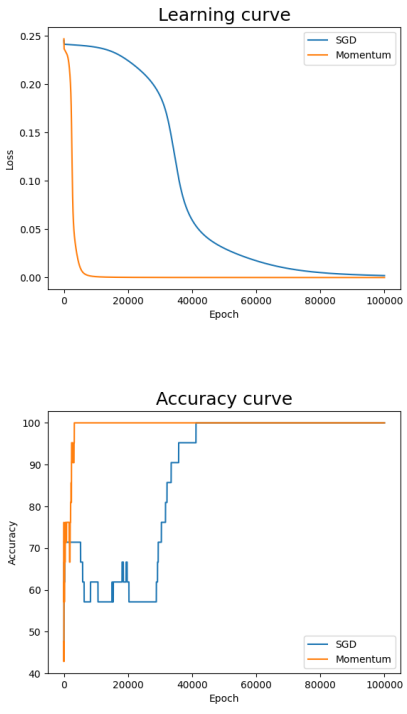
learning rate 太大會讓權重更新每次移動太多，跳過最佳解，使得訓練過程不穩定，loss 無法有效下降，導致模型無法收斂。如果 learning rate 太小則會讓權重更新每次移動太少，使得模型訓練緩慢，需要非常多 epoch 才能訓練，也可能因為移動太少，困在局部最佳解。

C. What is the purpose of weights and biases in a neural network?

Weights 可以改變神經元間彼此影響的大小程度，如果 weight 大，前一個節點就會對後一個節點有較大的影響力，也決定了神經元是要變大還是變小。Biases 可以為神經元提供平移，增加模型靈活程度。

6. Extra

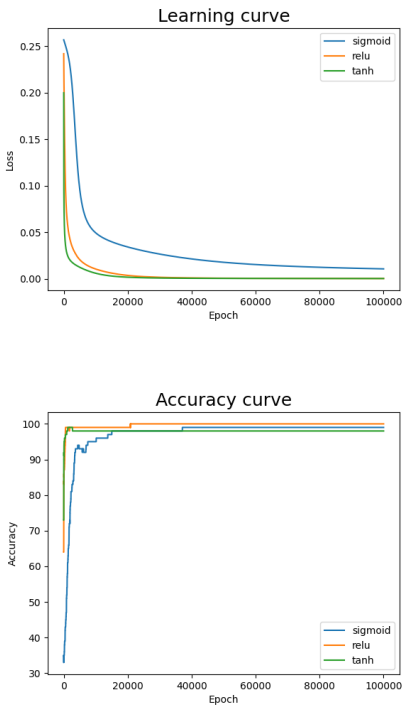
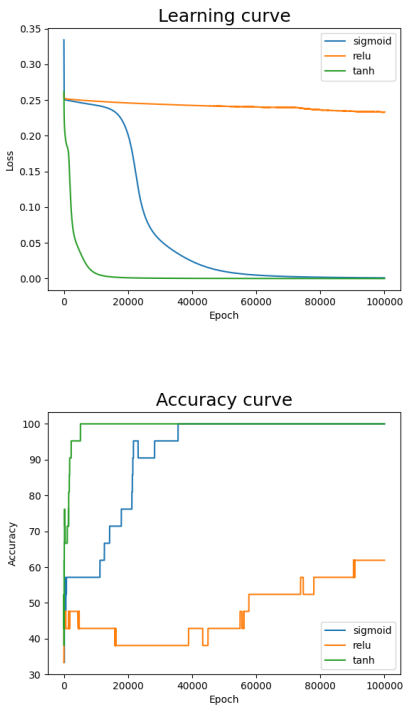
A. Implement different optimizers.

linear	XOR
	

我額外實作了 Momentum 這個 optimizer，累積過去的梯度用於後續

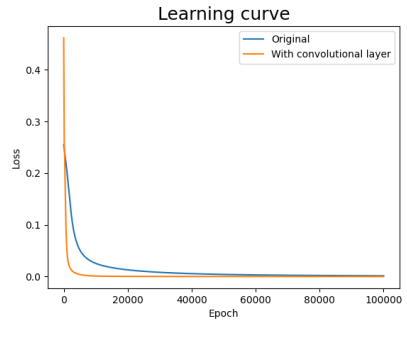
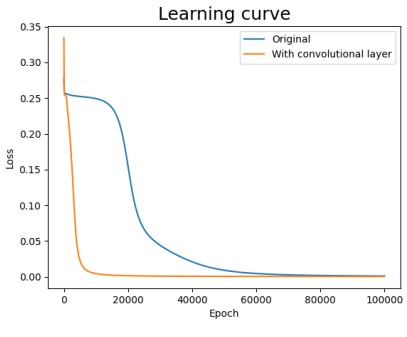
的梯度計算。相比 SGD，使用 Momentum 後，訓練速度有所提升。

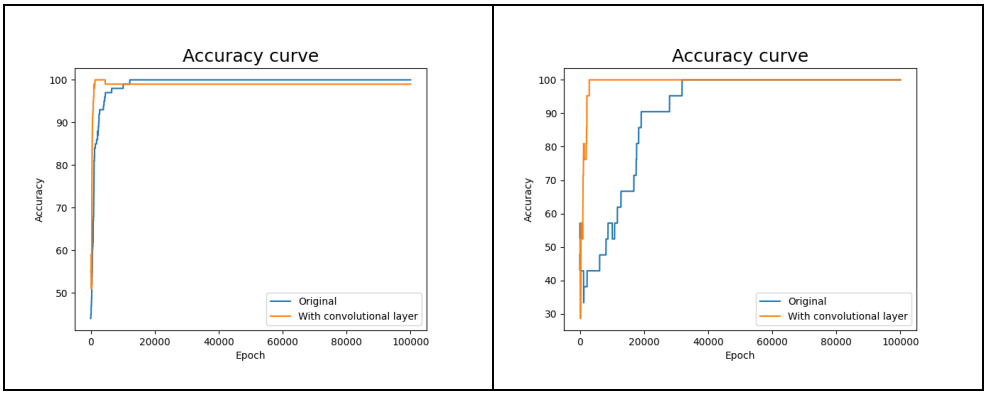
B. Implement different activation functions.

linear	XOR
	

我額外時做了 relu 和 tanh 兩個 activation functions，我發現 tanh 收斂最快，整體表現最好。Sigmoid 適合用於 linear，relu 適合用於 XOR。

C. Implement convolutional layers.

linear	XOR
	



我實作了 convolutional layer，放在兩個隱藏層中間，而隱藏層的節點數量改為 5 和 3，變為(2,5,3,1)，kernel size 為 3，並且有 bias。使用 convolutional layer 後，訓練速度快了很多，並且準確率依舊很高。